Approximating the Canadian Traveller Problem and Its Variations*

Yamming Huang and Chung-Shou Liao⋆
Department of Industrial Engineering and Engineering Management,
National Tsing Hua University, Hsinchu 30013, Taiwan
⋆csliao@ie.nthu.edu.tw

Abstract

Given a road map $G = (V, E)$ in which each edge $e \in E$ is associated with the time it takes for a traveller to traverse the edge. From an online perspective, the traveller is aware of the entire structure of G in advance; however, while some edges may be blocked by accidents during the trip, the problem will only become evident when the traveller reaches an end vertex incident to the blocked edge. Its objective is to design an efficient routing policy from a source to a destination under this condition of uncertainty. This problem, called the Canadian Traveller Problem *(CTP), was defined by Papadimitriou and Yannakakis.*

In this paper, we investigate a generalization of the CTP in which each edge e in E is associated with two possible distances. The traveller only learns about the actual distance cost of an edge e on arrival at one of its end vertices. We propose tight lower bounds of the problem when the number of traffic jams is a given constant k, and introduce a deterministic algorithm with a $\min\{r, 2k + 1\}$-ratio, which meets the proposed lower bound, where r is the worst-case performance ratio. We also study an extension to the metric Travelling Salesman Problem (TSP) and propose a touring strategy within an $O(\sqrt{k})$-competitive ratio. Finally, we discuss randomized strategies, in the hope of surpassing the barrier of obtaining better approximation algorithms.

Keywords. *Canadian traveller problem, competitive ratio, online algorithm*

*This work was supported by the National Science Council of Taiwan under Grant NSC100-2221-E-007-108-MY3.

1 Introduction

The Canadian Traveller Problem (CTP), and its variants can be considered as a natural assumption of some online routing problems. The CTP was initiated by Papadimitriou and Yannakakis [7] in 1991. The objective is to derive an adaptive strategy for travelling from a source s to a destination t so that the competitive ratio, which compares the distance traversed with that of the static s,t-shortest path in hindsight, is minimized. Papadimitriou and Yannakakis [7] proved that it is PSPACE-complete for the CTP to devise a strategy that guarantees a bounded competitive ratio.

For several years, there has been no significant progress in the development of competitive algorithms for this problem. Bar-Noy and Schieber [1] explored several variations of the CTP from the worst-case scenario perspective, where the objective is to find a static (offline) algorithm that minimizes the maximum travel cost [2]. They considered the k-CTP, in which the number of blockages is bounded from below by a given constant k. In addition, they discussed the Recoverable k-CTP, where each blocked edge is associated with a recovery time, which is not very long to reopen relative to the traversed time. Subsequently, Karger and Nikolova [6] investigated the stochastic CTP in special graph classes, and developed exact algorithms by applying techniques from the theory of Markov Decision Processes.

Recently, Westphal [11] proved that there are no deterministic online algorithms within a $(2k+1)$-competitive ratio for the k-CTP. The author designed a simple *reposition* algorithm that satisfies the lower bound, and also proposed a lower bound of $k+1$ for the competitive ratio of any randomized online algorithms. Xu *et al.* [12] developed two deterministic adaptive policies: a *greedy* strategy as well as a *comparison* strategy that incorporates the concept of reposition. The latter strategy also achieves the tight lower bound.

In this paper, we study a natural generalization of CTP, called the DOUBLE-VALUED GRAPH problem, which was initiated by Papadimitriou and Yannakakis [7]. The problem finds real applications in dynamic navigation systems used to avoid traffic congestion. Given a graph $G = (V, E)$ with a source s and a destination t in V, each edge e in E is associated with two possible distances: original $d(e)$ and jam $d^+(e)$, where d, $d^+ : E \to R^+$ and $d(e) < d^+(e)$, for each $e \in E$. A traveller only finds out which one of the two distances of an edge upon reaching an end vertex incident to the edge. The goal is to develop an adaptive strategy for traversing the graph from s to t with incomplete information about traffic conditions, so that the competitive ratio is minimized. This problem is also PSPACE-complete, as shown by a reduction from quantified SAT (QSAT) [7]. Here, for a graph G without any jammed edges, i.e., $d^+(e) = d(e)$, for any $e \in E$, the distance of the s,t-shortest path is denoted by $d(s,t)$. On the other hand, let the s,t-shortest path P consist of ℓ edges, and consider the distance of the path P in the worst case during the trip, denoted by $d^+(s,t,k)$. More precisely, given a constant bound k of the number of jammed edges, if $\ell < k$, $d^+(s,t,k) = d(s,t) + \sum_{e \in P}(d^+(e) - d(e))$; otherwise, let E' consist of the k jammed edges that have the maximum jam costs in P. Then, $d^+(s,t,k) = d(s,t) + \sum_{e \in E'}(d^+(e) - d(e))$.

Our contribution. The main results of this study are detailed below.

1. We provide tight lower bounds for deterministic and randomized algorithms for DOUBLE-VALUED GRAPH in terms of k and $r = \frac{d^+(s,t,k)}{d(s,t)}$ when the number of traffic jams is up to a given constant k.

2. We present a deterministic adaptive strategy with a $\min\{r, 2k+1\}$-competitive ratio that meets the proposed lower bound. The algorithm can also be applied directly to the Recoverable k-CTP that assumes the blocked edges are not found to be blocked again.

3. We study the uniform jam cost model of this problem, i.e., for every edge e, $d^+(e) = d(e) + c$, for a constant c, and derive a tight lower bound with an additive competitive ratio.

4. We investigate an extension of the k-CTP to the metric Travelling Salesman Problem. The goal is to design a tour of a set of vertices whereby the traveller visits each vertex and returns to the origin under the same uncertainty as the k-CTP, such that the competitive ratio is minimized. We propose a touring strategy within an $O(\sqrt{k})$-competitive ratio and show that the analysis is tight.

5. Finally, we discuss randomized algorithms for the k-CTP where there is a dearth of research. Westphal [11] proved the lower bound of $2k+1$ for the competitive ratio of any deterministic online algorithms. However, for randomized strategies, the lower bound on the competitive ratio is $k+1$. We exlpore the k-CTP in special cases for which an optimal randomized strategy exists, in the hope that the technique could surpass the barrier of obtaining better approximation algorithms for general graphs.

2 Preliminaries

We consider the DOUBLE-VALUED GRAPH problem with at most k traffic jams. Given a connected graph $G = (V, E)$ with a source s and a destination t, we denote the sequence of traffic jams in E learned by an online algorithm A during the trip as $S_i^A = (e_1, e_2, \ldots, e_i)$, where $1 \le i \le k$. Let $E_i^A = \{e_1, e_2, \ldots, e_i\} \subseteq E$, $1 \le i \le k$, consist of these jammed edges, and let E_k be the set of all jammed edges. In the following, the superscript A may be omitted without causing confusion. In addition, let $d : E \to R^+$ be the original distance function. The (traffic) jam distance function is $d^+ : E \to R^+$; that is, for each edge $e = (u,v) \in E$, $d(u,v) < d^+(u,v)$. Moreover, in the online problem, let $d_{E_i^A}(s,t)$ denote the travel cost from s to t, derived by an adaptive algorithm A that learns about traffic jam information E_i during the trip; and let $d_{E_k}(s,t)$ be the offline optimum from s to t under complete information E_k. We immediately have the following property [12], where $E_1 \subseteq E_2 \subseteq \ldots \subseteq E_k$.

$$d(s,t) \le d_{E_1}(s,t) \le \cdots \le d_{E_k}(s,t). \qquad (1)$$

We refer to [3, 8] and formally define the competitive ratio as follows: an online algorithm A is c^A-*competitive* for the DOUBLE-VALUED GRAPH problem if for any instances,

$$d_{E_i^A}(s,t) \le c^A \cdot d_{E_k}(s,t) + \varepsilon, \quad 1 \le i \le k,$$

where c^A and ε are constants.

Similar to the proof in [11], we propose tight lower bounds for DOUBLE-VALUED GRAPHS when we use deterministic and randomized algorithms, respectively. Due to space limitation, we omit some proofs in the following.

Lemma 2.1. *For the* DOUBLE-VALUED GRAPH *problem, there is no deterministic online algorithm with a competitive ratio less than* $\min\{r, 2k+1\}$ *when the number of jammed edges is up to a given constant* k.

Next, given the independent probabilities of traffic congestion on all the edges, we consider randomized strategies to solve this online problem.

Lemma 2.2. *For the* DOUBLE-VALUED GRAPH *problem, there is no randomized online algorithm with a competitive ratio less than* $\min\{r, k+1\}$ *when the number of jammed edges is up to a given constant* k.

Based on the proofs of Lemmas 2.1 and 2.2, we introduce two strategies: the *greedy algorithm* and the *reposition algorithm* [11, 12], which will be used later. We denote them as GA and RA, respectively.

Greedy Algorithm (GA): Starting at a vertex v (including the source s), the traveller selects the shortest path from v to t by using Dijkstra's algorithm [5] in a greedy manner, based on the current information E_i; that is, the distance cost of the path is $d_{E_i}(v,t)$. Note that if all the k jammed edges are known at the outset, the cost of the path derived by GA from the source s is the same as that of the offline optimum, $d_{E_k}(s,t)$.

Reposition Algorithm (RA): The traveller begins at the source s and follows the s,t-path with the cost $d(s,t)$. When the traveller learns about a jammed edge on the path to t, he/she returns to s and takes the s,t-path with the cost $d_{E_i}(s,t)$ based on the current information E_i. The traveller repeats this strategy until he/she arrives at t.

3 Double-valued Graph

In this section, we present a deterministic algorithm called GR, which combines GA with RA to solve the DOUBLE-VALUED GRAPH problem. Its ratio meets the proposed lower bound. For convenience, let $e_i = (v_i, v_{i'})$, $1 \le i \le k$ be a jammed edge learned of by the traveller; and let v_i be the first end vertex of the jammed edge e_i that the traveller visits during the trip.

This algorithm mainly consists of three routing policies when the traveller arrives at v_i. The first strategy is to follow a path from v_i to t derived by GA, irrespective of whether the traveller finds a new jammed edge. The second strategy for the traveller is to move from v_i to t via GA until he/she finds a jammed edge; and the last strategy is to move from v_i to t via RA until the traveller finds a jammed edge. Note that the ratio r has to be updated while the traveller is learning about a new jammed edge. In addition, because $2(k-i)+1$ decreases during the trip, $r \le \frac{d^+(s,t,k)}{d(s,t)}$ once the traveller selects the first strategy, i.e., following a path derived by GA.

Lemma 3.1 considers the competitive ratio of the GR algorithm under the condition $r > 2(k-i)+1$, in which the traveller selects the last two routing policies. Next, we combine the scenario with the first strategy, and Theorem 3.2 follows.

Lemma 3.1. *If the ratio* $r = \frac{d^+(v_i,t,k-i)}{d_{E_i}(s,t)} > 2(k-i)+1$, *for each* i *during the whole trip, then the total distance cost of GR satisfies the following property, where* $1 \le i \le k$, *provided there is a set of jammed edges* E_i .

$$d_{E_i^{GR}}(s,t) \le \begin{cases} (i+1)\cdot d_{E_i}(s,t), \\ \textit{if the traveller uses GA at } v_i; \\ (2i+1)\cdot d_{E_i}(s,t), \\ \textit{if the traveller uses RA at } v_i. \end{cases}$$

Proof By induction on the number of jammed edges learned by the traveller, the proof is trivial for the case $i=0$. Consider $i=1$. The traveller walks from v_1 to t using GA when $d(s,t) + d_{E_1}(v_1,t) \le (1+1)\cdot d_{E_1}(s,t)$. Thus, the travel cost is $d_{E_1^{GR}}(s,t) \le d(s,t) + d_{E_1}(v_1,t) \le 2d_{E_1}(s,t)$ for the case $i=1$. Otherwise, the traveller uses RA at v_1; that is, the traveller returns to s, and takes the path with the cost $d_{E_1}(s,t)$. Thus, by Equation (1), the total cost is at most $2d(s,t) + d_{E_1}(s,t) \le (2+1)\cdot d_{E_1}(s,t)$.

Assume the statement holds for $i \le \ell$. We divide the case $i = \ell+1$ into two parts:

Case 1: The traveller chooses GA at $v_{\ell+1}$; that is, $d_{E_\ell^{GR}}(s,t) + d_{E_{\ell+1}}(v_{\ell+1},t) \le ((\ell+1)+1)\cdot d_{E_{\ell+1}}(s,t)$. Hence, the statement follows, because

$$\begin{aligned} d_{E_{\ell+1}^{GR}}(s,t) &\le d_{E_\ell^{GR}}(s,t) + d_{E_{\ell+1}}(v_{\ell+1},t) \\ &\le (\ell+2)\cdot d_{E_{\ell+1}}(s,t). \end{aligned}$$

Algorithm 1: Greedy & Reposition Algorithm (GR)

Input : $G = (V, E)$, $d : E \to R^+$, $d^+ : E \to R^+$ and a constant k;
Output : A route from s to t;

Initialize $i = 0$ and let $v_0 = s$ and $E_0 = \emptyset$;
while the traveller does not arrive at t **do**
    Let $r = \frac{d^+(v_i,t,k-i)}{d_{E_i}(s,t)}$;
    if $r \leq 2(k-i)+1$ **then**
        the traveller follows a path from v_i to t derived by GA; ▷ 1^{st} strategy
    else
        if $d_{E^{GR}_{i-1}}(s,t) + d_{E_i}(v_i,t) \leq (i+1) \cdot d_{E_i}(s,t)$ **then**
            the traveller moves from v_i to t via GA until he/she finds a jammed edge; ▷ 2^{nd} strategy
        else
            the traveller moves from v_i to t via RA until he/she finds a jammed edge; ▷ 3^{rd} strategy
        end if
    end if
    Let $i = i+1$ and let the new jammed edge be $e_i = (v_i, v_{i'})$ during the trip;
end while

Case 2: The traveller chooses RA at $v_{\ell+1}$. Consider two scenarios when the traveller was at v_ℓ. If the traveller used GA at v_ℓ, then $d_{E^{GR}_\ell}(s,t) \leq (\ell + 1) \cdot d_{E_\ell}(s,t)$ by the induction hypothesis. Similarly, the traveller would go back to s from $v_{\ell+1}$ and take the path with the cost $d_{E_{\ell+1}}(s,t)$. Therefore, we have

$$\begin{aligned} d_{E^{GR}_{\ell+1}}(s,t) &\leq 2 \cdot d_{E^{GR}_\ell}(s,t) + d_{E_{\ell+1}}(s,t) \\ &\leq 2(\ell+1) \cdot d_{E_\ell}(s,t) + d_{E_{\ell+1}}(s,t) \\ &\leq (2(\ell+1)+1) \cdot d_{E_{\ell+1}}(s,t) \end{aligned}$$

by Equation (1). On the other hand, if the traveller used RA at v_ℓ, then by the induction hypothesis, $d_{E^{GR}_\ell}(s,t) \leq (2\ell+1) \cdot d_{E_\ell}(s,t)$. Because the traveller started again from s and took the path with the cost $d_{E_\ell}(s,t)$ after learning about the jammed edge at v_ℓ, the distance cost of the path that the traveller used to return to s from $v_{\ell+1}$ will not be larger than $d_{E_\ell}(s,t)$. Thus, we have

$$\begin{aligned} d_{E^{GR}_{\ell+1}}(s,t) &\leq d_{E^{GR}_\ell}(s,t) + d_{E_\ell}(s,t) + d_{E_{\ell+1}}(s,t) \\ &\leq (2\ell+1) \cdot d_{E_\ell}(s,t) + d_{E_\ell}(s,t) \\ &\quad + d_{E_{\ell+1}}(s,t) \\ &\leq (2(\ell+1)+1) \cdot d_{E_{\ell+1}}(s,t). \end{aligned}$$

The proof is complete. □

The above lemma shows the competitive ratio without using the first strategy, i.e., following a path derived by GA, irrespective of whether the traveller finds a jammed edge. We remark that if the traveller does not know the bound k of the number of jammed edges, that is, r cannot be derived initially, then the last two routing policies of the GR algorithm can obtain a $(2k+1)$-competitive ratio by Lemma 3.1.

We prove that the GR algorithm is $\min\{r, 2k+1\}$-competitive if the number of jammed edges is bounded from below by a given constant k.

Theorem 3.2. *For the* Double-valued Graph *problem, the competitive ratio of GR is at most $\min\{r, 2k+1\}$ when the number of jammed edges is up to a given constant k, where $r = \frac{d^+(s,t,k)}{d(s,t)}$ initially, and r might decrease during the trip.*

Proof First, if the ratio $r = \frac{d^+(s,t,k)}{d(s,t)} \leq 2k+1$ initially, the traveller follows a path from s to t derived by GA. The competitive ratio is $\frac{d^+(s,t,k)}{d_{E_k}(s,t)} \leq \frac{d^+(s,t,k)}{d(s,t)} = r \leq 2k+1$. Otherwise, the traveller follows GA or RA until he/she discovers a jammed edge. Assume that $r = \frac{d^+(v_i,t,k-i)}{d_{E_i}(s,t)} > 2(k-i)+1$ when the traveller learns about each jammed edge $e_i = (v_i, v_{i'})$ during the trip $1 \leq i \leq k$. That is, the traveller will never use the first strategy. Therefore, the competitive ratio is within $2k+1$ by Lemma 3.1.

On the other hand, if $r \leq 2(k-j)+1$ while learning about a jammed edge $e_j = (v_j, v_{j'})$, for some j, the traveller will follow a path from v_j to t derived by GA. The travel cost is at most $d_{E^{GR}_{j-1}}(s,t)$ before the traveller arrives at v_j. Then the total distance cost can be formulated as follows:

$$\begin{aligned} d_{E_j^{GR}}(s,t) &\le d_{E_{j-1}^{GR}}(s,t) + d^+(v_j,t,k-j) \\ &\le (2(j-1)+1)\cdot d_{E_{j-1}}(s,t) \\ &\quad + (2(k-j)+1)\cdot d_{E_j}(s,t) \\ &\le 2k\cdot d_{E_k}(s,t). \end{aligned}$$

The second inequality holds by Lemma 3.1. □

Regarding the time complexity analysis, the number of iterations in the while loop, i.e., the number of updates for the ratio r, is at most k. Each of the three routing strategies can apply Dijkstra's algorithm [5] to devise a path from s or v_i to t, for some i. In addition, RA just takes the original s, v_i-path when the traveller needs to return to s from v_i. Thus, for a given constant k, the running time is a constant factor times $D(n)$, where n is the order of a graph G, and $D(n)$ is the running time of Dijkstra's algorithm.

The GR algorithm can also be extended to the Multiple-valued Graph problem in which each edge is associated with more than two possible distances. We regard the largest distance of each edge e as $d^+(e)$, and then $r = \frac{d^+(s,t,k)}{d(s,t)}$ is defined similarly. GR performs in a similar way to travel from s to t. Therefore, the competitive ratio remains the same.

Corollary 3.3. *For the* Multiple-valued Graph *problem, there is a* $\min\{r, 2k+1\}$*-competitive algorithm when the number of traffic jams is up to a given constant* k.

In addition, we consider the Recoverable k-CTP in which each blocked edge e is associated with a recovery time $r(e)$ to reopen. In this online problem, it is assumed that the blocked edges will not be blocked again. An instance I of the Recoverable k-CTP can be transformed into an instance I' of the Double-valued Graph problem by letting $r(e)$ be represented in terms of the distance, and letting $d^+(e) = d(e) + r(e)$ for every edge $e \in E$ in I'.

The GR algorithm can then be modified as follows. When learning about a jammed edge e_i, the traveller decides if he/she will traverse the edge e_i via GR. If the traveller traverses the jammed edge e_i, then $E_i = E_{i-1} \cup \{e_i\}$, and after passing through the edge e_i, e_i will be reset to its original distance $d(e_i)$. All the other operations are performed as described earlier, and the next corollary follows immediately.

Corollary 3.4. *For the Recoverable* k*-CTP, there is a* $\min\{r, 2k+1\}$*-competitive algorithm when the number of blockages is bounded from below by a given constant* k.

Su *et al.* [9] considered the Recoverable k-CTP and proposed two policies: a *greedy* strategy and a *waiting* strategy. They also investigated the problem and its variants in special road networks [10]. When the number of blocked edges is up to a given constant k, Table 1 compares the competitive ratios of the previous strategies with the ratio of GR under different scenarios. For example, if $r(e) \le \alpha \cdot d(e)$, $\alpha > 0$, i.e., $d^+(e) \le (1+\alpha)d(e)$ in the Double-valued Graph problem, then $r = \frac{d^+(s,t,k)}{d(s,t)} \le 1+\alpha$. If $\alpha \le 2k$, GR will initially follow the path derived by GA. Thus, we have the competitive ratio $r \le 1+\alpha$. Otherwise, if $\alpha > 2k$, the competitive ratio of GR is at most $2k+1$ by Lemma 3.1. Besides, if $r(e) \le \alpha \cdot d(s,t)$, it implies that $r = \frac{d^+(s,t,k)}{d(s,t)} \le \alpha k + 1$. Therefore, if $\alpha \le 2$, GR will follow a path derived by GA, and we have the competitive ratio $r \le \alpha k + 1$. Otherwise, if $r(e) > 2d(s,t)$, then $r > 2k+1$, and the competitive ratio of GR is at most $2k+1$ by Lemma 3.1. Note that the results show that the GR approach is at least as good as the previous results in [9] when the number of blockages is bounded from below by a given constant k.

4 The Uniform Jam Cost Model

In this section, we study the uniform jam cost model. Suppose the jam cost of each edge e is a constant c, i.e., $d^+(e) = d(e) + c$. An online algorithm A is said to be a c^A*-additive competitive algorithm* for the Double-valued Graph problem if for any instances,

$$d_{E_i^A}(s,t) \le d_{E_k}(s,t) + c^A, \quad 1 \le i \le k,$$

where c^A is a constant. We propose a tight lower bound below.

Lemma 4.1. *For the uniform jam cost model of the* Double-valued Graph *problem with at most* k *traffic jams, there is no deterministic online algorithm within a* kc*-additive competitive ratio; that is, given a uniform jam cost* c*, the derived solution cannot be better than* $d_{E_k}(s,t) + kc$.

By Lemma 4.1, no deterministic algorithm can derive a better additive competitive ratio than kc. Actually, the traveller can use a very straightforward algorithm to achieve the lower bound: following the s,t-path with the cost $d(s,t)$, irrespective of whether the traveller finds jammed edges.

recovery time	greedy strategy [9]	waiting strategy [9]	GR
$r(e) \le d(e)$	2^k	2	$r \le 2$
$r(e) \le \alpha \cdot d(e)$	$(1+\alpha)^k$	$1+\alpha$	$\min\{1+\alpha, 2k+1\}$
$r(e) \le d(s,t)$	2^k	$k+1$	$r \le k+1$
$r(e) \le \alpha \cdot d(s,t)$	$(1+\alpha)^k$	$\alpha k+1$	$\min\{\alpha k+1, 2k+1\}$

Table 1: Comparison of the GR results and the results reported in [9], where $\alpha > 0$

However, it is possible to slightly improve the ratio under some conditions.

For instance, if $c \ge 2\delta \cdot d(s,t)$ for some $\delta > 1$, we let a threshold be $\frac{c}{2\epsilon}$, for a constant $1 < \epsilon < \delta$. Assume there is a nonempty subset of jammed edges E_i, such that $d_{E_i}(s,t) \le \frac{c}{2\epsilon}$ for some $i \le k$, when the traveller learns about jammed edges. Based on this assumption, when $d_{E_j}(s,t) \le \frac{c}{2\epsilon}$, the traveller will use RA; however, when $d_{E_j}(s,t) > \frac{c}{2\epsilon}$, the traveller will use GA until he/she arrives at t. Thus, we let e_ℓ be the first jammed edge, such that $d_{E_\ell}(s,t) > \frac{c}{2\epsilon}$, if any. That is,

$$\begin{cases} d_{E_j}(s,t) \le \frac{c}{2\epsilon}, & \text{if } 1 \le j < \ell; \\ \triangleright \text{ the traveller uses } RA & \\ d_{E_j}(s,t) > \frac{c}{2\epsilon}, & \text{if } \ell \le j \le k. \\ \triangleright \text{ the traveller uses } GA & \end{cases}$$

Then, the total travel cost can be formulated as follows:

$$\begin{aligned} & 2 \cdot d(s,t) + \ldots + 2 \cdot d_{E_{\ell-2}}(s,t) + d_{E_{\ell-1}}(s,t) \\ & + (k-\ell+1)c \\ \le\ & 2(\ell-1) \cdot \frac{c}{2\epsilon} + d_{E_{\ell-1}}(s,t) + (k-\ell+1)c \\ \le\ & d_{E_k}(s,t) + kc - (\ell-1)(1-\frac{1}{\epsilon})c. \end{aligned}$$

If there is no such ℓ, the traveller uses RA until he/she arrives at t. The above equation implies that the total travel cost is at most $2k(\frac{c}{2\epsilon}) + d_{E_k}(s,t) = \frac{kc}{\epsilon} + d_{E_k}(s,t)$.

5 k-CTP for Metric TSP

In this section, we extend the k-CTP to the metric travelling salesman problem (TSP). We refer to [12] and present several natural assumptions in the following. Given a complete graph $G = (V, E)$ of order n, i.e., $|V| = n$, the graph is still connected even if blocked edges are removed. Otherwise, the traveller would not be able to visit all the vertices in G. Moreover, we assume that the upper bound of the number of blockages is $k < n-1$ for the same reason. In addition, the traveller will only find a blockage upon reaching an end vertex of the blocked edge, and the state of the edge will not change again after the traveller learns about the blockage. The objective is to design a tour that enables the traveller to visit every vertex and return to the origin under the uncertainty, such that the competitive ratio is minimized. Note that the problem allows the traveller to visit a vertex more than once because the blocked edges may cause the assumption to occur.

The rationale behind our approach is that the traveller will visit as many vertices as possible via alternative routes based on a tour derived by Christofides' algorithm [4], denoted by $P : s = v_1 - v_2 - \ldots - v_n - s$. Assume the traveller takes the tour P, which is supposed to visit all the vertices in V in m^* rounds. Let V_m be the set of unvisited vertices in the m^{th} round, $1 \le m \le m^*+1$, and $V_1 = V \setminus \{s\}$. Then, we have $V \supset V_1 \supseteq V_2 \supseteq \ldots \supseteq V_{m^*} \supset V_{m^*+1} = \emptyset$. In addition, we denote a path over V_m as $P_m : v_{m,0} - v_{m,1} - \ldots - v_{m,|V_m|}$ when the traveller tries to visit the vertices in V_m in the m^{th} round by using the Cyclic Routing (CR) algorithm (see Algorithm 2). Note that $v_{m,0}$ is the last vertex visited in the $(m-1)^{th}$ round when $m > 1$, i.e., $v_{m,0} \notin V_m$.

As mentioned earlier, we let E_i^{CR} consist of the blocked edges revealed by the CR algorithm, $1 \le i \le k$. We decompose E_i^{CR} into m^* subsets called $E_{m,i}^{CR}$. Each subset is learned by the traveller in the m^{th} round, $1 \le m \le m^*$, i.e., $E_i = \{e_1, \ldots, e_i\} = \bigcup_{m=1}^{m^*} E_{m,i}^{CR}$. For convenience, we use E'_m instead of $E_{m,i}^{CR}$; and we denote a path p from u to v as $p : u \sim v$.

Lemma 5.1. *The traveller can visit at least one vertex in V_m by using CR in the m^{th} round, $1 \le m \le m^*$; that is, $|V_1| > |V_2| > \ldots > |V_m| > |V_{m+1}| > \ldots > |V_{m^*}|$.*

Proof Assume the traveller cannot visit any vertex in V_m for some m. The traveller starts at $v_{m,0}$. Consider the case where $m = 1$ or $v_{m,0} = v_{m-1,|V_{m-1}|}$. Because the traveller does not use the DETOUR procedure to visit any vertices in

Algorithm 2: Cyclic Routing Algorithm (CR)

Input : $G = (V, E)$ of order n, $d : E \to R^+$ and a constant k;
Output : A tour P^* that visits every vertex in V;

Use Christofides' algorithm to find a tour $P : s = v_1 - v_2 - \ldots - v_n - s$ that visits every vertex exactly once from s and return to s, and initialize $P^* = \emptyset$, $m = 1$, $V_1 = V \setminus \{s\}$, and $v_{1,0} = s$;
while $V_m \neq \emptyset$ **do**
  Let a path over V_m be $P_m : v_{m,0} - v_{m,1} - \ldots - v_{m,|V_m|}$;
  if $m = 1$ or $v_{m,0} = v_{m-1,|V_{m-1}|}$ **then**
    Perform **procedure** DETOUR in the same direction as that in the $(m-1)^{th}$ round or in a clockwise direction when $m = 1$;
    if $V_{m+1} = V_m$ **then**
      Perform **procedure** DETOUR in the opposite direction;
    end if
  else
    Perform **procedure** DETOUR in the opposite direction to that in the $(m-1)^{th}$ round;
  end if
  $m = m + 1$;
end while
The traveller returns to s directly or finds a previously visited vertex u such that $v_{m,0} - u - s$ is not blocked and traverse the alternative route;
$P^* = P^* \cup \{v_{m,0} \sim s\}$;
return P^*;

procedure DETOUR
  Let $V_{m+1} = V_m$ and $E'_m = \emptyset$;
  for $i = 1$ to $|V_m|$ **do**
    if $v_{m,i-1} - v_{m,i}$ is not blocked or there is an internal visited vertex u in $v_{m,i-1} \sim v_{m,i}$ of P such that $v_{m,i-1} - u - v_{m,i}$ is not blocked **then**
      $V_{m+1} = V_{m+1} \setminus \{v_{m,i}\}$ and $P^* = P^* \cup \{v_{m,i-1} \sim v_{m,i}\}$;
    else
      Let $v_{m,i} = v_{m,i-1}$;
    end if
    $E'_m = E'_m \cup \{e\}$, for each new blockage e the traveller just learned;
  end for
end procedure

V_m, there is a blockage for each edge $(v_{m,0}, v_{m,i})$, $1 \leq i \leq |V_m|$. There is also a blocked edge on any alternative route $v_{m,0} - u - v_{m,i}$, $u \in V \setminus V_m$. The procedure also scans in the opposite direction to check if there is a detour $v_{m,0} - u - v_{m,|V_m|}$. Hence, the total number of blockages learned is at least $|V_m| + (n - |V_m| - 1) = n - 1 > k$, which is a contradiction.

Otherwise, $v_{m,0} \neq v_{m-1,|V_{m-1}|}$. Hence, we have $v_{m-1,|V_{m-1}|} \in V_m$ because $v_{m,0}$ is the last vertex visited in the $(m-1)^{th}$ round. The procedure found a blockage for any detour $v_{m,0} - u - v_{m-1,|V_{m-1}|}$ in the $(m-1)^{th}$ round. Besides, the traveller would traverse the path P_m in the opposite direction to that in the $(m-1)^{th}$ round. Similarly, the procedure scans all the vertices from $v_{m,0}$ to at least $v_{m-1,|V_{m-1}|}$ in order to visit vertices in V_m directly or via alternative routes. Thus, the number of blockages learned is at least $n - 1 > k$. This also contradicts the assumption; thus, the proof is complete. □

Based on the above key lemma, the tour P^* derived by the CR algorithm traverses all the vertices in V until $V_{m^*+1} = \emptyset$; that is, P^* is a feasible solution. The next lemmas follow immediately.

Lemma 5.2. *$E'_i \cap E'_j = \emptyset$ for any $1 \leq i < j \leq m^*$.*

Proof Consider the path P_j over the set of unvisited vertices V_j, $1 < j \leq m^*$. Because the traveller learned about all the edges incident to an internal visited vertex u in the previous $j - 1$ rounds while looking for alternative routes in the j^{th} round, each new blockage has both of its end vertices in V_j. In addition, if a blocked edge $e \in E'_i$, $1 \leq i < j$, was learned in the i^{th} round and one of its end vertices is in V_j, then the other end vertex is in $V_i \setminus V_j$; otherwise, it would be impossible for the traveller to find the blockage e whose two end vertices are unvisited. Thus, any blockage in E'_j does not appear in E'_i. □

Lemma 5.3. *The number of new blockages learned in the m^{th} round is not less than the number of unvisited vertices in the $(m+1)^{th}$ round, i.e., $|E'_m| \geq |V_{m+1}|$, $1 \leq m \leq m^*$.*

Proof Based on the proof of Lemma 5.2, the end vertices of each blockage learned in E'_m are in V_m. By Lemma 5.1, at least one new vertex, say v, is visited by the DETOUR procedure in the m^{th} round; hence, the traveller tries to explore unvisited vertices from $v_{m,0}$ and v in the m^{th} round. During the process, at most one unvisited vertex in V_m will be moved to V_{m+1} when a blocked edge is found in E'_m. Therefore, $|E'_m| \geq |V_{m+1}|$. □

We develop the CR strategy based on a tour P derived by Christofides' algorithm for the original metric TSP. It combines the minimum spanning tree of G with the minimum weight perfect matching on the vertices with odd degree in the tree to obtain a Hamiltonian tour with a $\frac{3}{2}$-approximation ratio if G satisfies the triangle inequality property. Let OPT be the offline optimum of the k-CTP for the metric TSP. Obviously, OPT cannot be less than the optimum of the original TSP.

Lemma 5.4. *When the traveller uses CR to traverse P_m in the m^{th} round, $1 \leq m \leq m^*$, the travel cost is not larger than $3OPT$; and for the $(m^*+1)^{th}$ round, the travel cost is OPT at most.*

Proof Because of the triangle inequality, the traveller traverses P_m of length at most $2|P|$ in each of the previous m rounds. Thus, the travel cost is at most $3OPT$ in a single round. Consider the $(m^*+1)^{th}$ round with $V_{m^*+1} = \emptyset$. Because $k < n-1$, the traveller can either traverse the edge $(v_{m+1,0}, s)$ directly or stop at a visited vertex to return to s. The distance cost of each edge is at most $\frac{1}{2}OPT$ because of the triangle inequality, so the cost in the last round is at most OPT. □

Theorem 5.5. *The k-CTP for the metric TSP can be approximated within an $O(\sqrt{k})$-competitive ratio when the number of blockages is bounded from below by k.*

Proof By Lemma 5.2 and $|\bigcup_{m=1}^{m^*} E'_m| \leq k$, we have

$$|E'_1| + |E'_2| + \cdots + |E'_{m^*}| \leq k,$$

because any two E'_i and E'_j are disjoint, $1 \leq i < j \leq m^*$. In addition, by Lemma 5.3, we have

$$\begin{aligned} &|V_2| + |V_3| + \cdots + |V_{m^*+1}| \\ &\leq |E'_1| + |E'_2| + \cdots + |E'_{m^*}| \leq k \end{aligned} \tag{2}$$

In the worst case, $|V_m \setminus V_{m+1}| = 1$, $1 \leq m \leq m^*$ by Lemma 5.1; that is, $|V_{m^*+1}| = 0$, $|V_{m^*}| = 1$, $\ldots$, $|V_2| = m^* - 1$. By Equation (2), the number of rounds is at most $O(\sqrt{k})$, which can be derived as follows:

$$\begin{aligned} &\frac{(1+(m^*-1))(m^*-1)}{2} \leq k \\ \Rightarrow &\frac{1-\sqrt{1+8k}}{2} \leq m^* \leq \frac{1+\sqrt{1+8k}}{2} \\ \Rightarrow &m^* \leq \left\lfloor \frac{1+\sqrt{1+8k}}{2} \right\rfloor \end{aligned}$$

Let $d_{E_k^{CR}}(s,s)$ denote the travel cost of the tour P^* derived by CR, when the number of blocked edges is up to k. Then, by Lemma 5.4, the total cost of the tour is

$$d_{E_k^{CR}}(s,s) \leq (3m^*+1)OPT, \text{ where } m^* = O(\sqrt{k})$$

Hence the competitive ratio is within $O(\sqrt{k})$. □

The next corollary shows the tightness of the analysis of the competitive ratio.

Corollary 5.6. *The analysis of an $O(\sqrt{k})$-competitive ratio of the CR algorithm is tight.*

6 Randomized algorithm

In this section, we investigate the k-CTP in special cases for which an optimal randomized strategy exists, in the hope that the randomized technique could improve the currently best deterministic algorithm for general graphs. Consider a special road network as follows. Given a graph $G = (V, E)$ with a source s and a destination t in V, for each vertex v, $v \notin \{s,t\}$, there is exactly one v,t-path in G. That is, $G' = (V \setminus \{s\}, E')$ is a tree, where $E' = E \setminus \{e = (s,v) \mid v \in V\}$. More precisely, the special road network G consists of a source vertex s, a tree with root t, and edges connecting s to each leaf of the tree. Moreover, we assume that the distance cost of each s,t-path

Algorithm 3: Randomized Reposition Algorithm (RR)

Input : $G = (V, E)$ in $\mathtt{T}$, $d : E \to R^+$, and a constant k;
Output : A route from s to t;

1: Assign the probabilities equally among every child of the root t, and sequentially repeat the argument for each descendant of t until the leaves in a top-down manner;
2: The traveller randomly chooses an s,t-path via the probabilities and leaves for t following the path;
3: **while** the traveller does not arrive at t **do**
4:     Let the blocked edge learned by the traveller at v_i be $e = (v_i, v_j)$;
5:     **if** there is an s,t-path through the vertex v_j **then**
6:         the traveller returns to s and reassign the probabilities for the subtree rooted at v_j as Step 1;
7:         the traveller randomly selects an s,t-path through v_j via the probabilities;
8:     **else**
9:         the traveller returns to s and reassign the probabilities for G as Step 1 without considering
10:         the subtree rooted at v_j;
11:     **end if**
12: **end while**

in G is identical. Let $\mathtt{T}$ denote the graph class of such special road networks.

The key concept of our algorithm for the k-CTP in graphs in $\mathtt{T}$ is to incorporate randomized operations into the reposition strategy and sequentially explore subtrees of the underlying tree subgraph of a given special graph. We claim that the RR algorithm is an optimal randomized strategy for the k-CTP in the special graph class $\mathtt{T}$. That is, the competitive ratio of the RR algorithm meets the lower bound of $k+1$ for the competitive ratio of any randomized online algorithm, proposed in previous work [11]. Note that the lower bound can be proved in a similar manner.

For the k-CTP in a given graph $G = (V, E)$ in $\mathtt{T}$, there is at least one s,t-path without blockages. We consider the expected total cost when the traveller traverses other paths using the RR algorithm. Let the s,t-path without blockages be $P : s = v_1^* - v_2^* - \ldots - v_m^* = t$ and the number of children of a vertex v_j^* be c_j in the underlying tree subgraph of G, i.e, v_j^* has its children $v_{j,1}, v_{j,2}, \ldots, v_{j,c_j}$. Without loss of generality, assume the last child of each v_j^*, i.e., v_{j,c_j}, $2 \le j \le m$, lies on the path P. Besides, suppose each subtree rooted at $v_{j,\ell}$, $1 \le \ell \le c_j - 1$, have $b_{j,\ell}$ blockages. Note that the malicious adversary should not assign blockages to any edge $(s, v) \in E$, as mentioned earlier. Thus, $\sum_{j=3}^{m} \sum_{\ell=1}^{c_j-1} b_{j,\ell} = k$.

Theorem 6.1. *For the k-CTP in the special graph class $\mathtt{T}$, there exists an optimal $(k+1)$-competitive randomized algorithm.*

Proof. We prove the statement from a top-down perspective in the underlying tree subgraph of a given graph G in $\mathtt{T}$. When the traveller reaches the children of $v_m^* = t$, consider the probability of selecting an s,t-path via some child $v_{m,i}$ of v_m^*. Based on Step 1, if the traveller chooses an s,t-path via $v_{m,i}$ rather than $v_{m,\ell}$, $\ell \neq i$ in the beginning, the probability is $\frac{1}{c_m}$. If $v_{m,i}$ is the second choice, then the probability is $\frac{c_1-2}{c_1} \times \frac{1}{c_1-1}$ because the traveller cannot select v_{m,c_m}; otherwise, the algorithm ends. Repeat the argument and the total probability of traversing a path through $v_{m,i}$ is $\frac{1}{c_1} + \frac{c_1-2}{c_1} \times \frac{1}{c_1-1} + \ldots + \frac{c_1-2}{c_1} \times \ldots \times \frac{1}{3} \times \frac{1}{2}$. Note that the strategy will not explore other subtrees rooted at $v_{m,\ell}$, $\ell \neq i$, until the traveller traverse all the paths through $v_{m,i}$. Thus, for each $v_{m,\ell}$, $\ell \neq c_m$, the expected travel cost expended before selecting a path via v_{m,c_m} is at most $(\frac{1}{c_1} + \frac{c_1-2}{c_1} \times \frac{1}{c_1-1} + \ldots + \frac{c_1-2}{c_1} \times \ldots \frac{1}{3} \times \frac{1}{2}) \times b_{m,\ell} \times 2d_{E_k}(s,t) = b_{m,\ell} \cdot d_{E_k}(s,t)$. Similarly, when the traveller reaches the children of v_j^*, $3 \le j \le m-1$, the expected travel cost is at most $b_{j,\ell} \cdot d_{E_k}(s,t)$, for every $v_{j,\ell}$, $\ell \neq c_j$, before the traveller selects a path via v_{j,c_j}. Hence, the expected total cost of the RR algorithm is $(k+1)d_{E_k}(s,t)$ and its competitive ratio achieves the lower bound for any randomized online algorithms in the special graph class $\mathtt{T}$. The proof is complete. □

7 Conclusion

In this paper, we have studied the Double-valued Graph problem, which is a generalization of k-CTP, when the number of traffic jams is up to a given constant k. We have presented tight lower bounds and an adaptive algorithm that can satisfy the lower bound. In addition, we have extended the algorithm to the Recoverable k-CTP. We have also derived a lower bound with an additive competitive ratio for the uniform jam cost model. Moreover, we have considered the k-CTP for the metric TSP and provided an $O(\sqrt{k})$-competitive ratio algorithm. Finally, we have explored the possibilities of obtaining better approximation results via randomized strategies.

References

[1] A. Bar-Noy and B. Schieber. The Canadian traveller problem. In *Proc. of the 2nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, (1991), pp. 261–270.

[2] S. Ben-David and A. Borodin. A new measure for the study of online algorithms. *Algorithmica*, (1994), 11(1), pp. 73–91.

[3] A. Borodin and R. El-Yaniv. Online computation and competitive analysis. Cambridge University Press, Cambridge, (1998).

[4] N. Christofides. Worst-case analysis of a new heuristic for the traveling salesman problem. Technical report, Graduate School of Industrial Administration, Carnegie-Mellon University, (1976).

[5] E.W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, (1959), 1(1) pp. 269–271.

[6] D. Karger and E. Nikolova. Exact algorithms for the Canadian traveller problem on paths and trees. Technical report, MIT Computer Science & Artificial Intelligence Lab, (2008). URL http://hdl.handle.net/1721.1/40093.

[7] C.H. Papadimitriou and M. Yannakakis. Shortest paths without a map. *Theoretical Computer Science*, (1991), 84(1), pp. 127–150.

[8] D. Sleator and R.E. Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, (1985), 28, pp. 202–208.

[9] B. Su and Y.F. Xu. Online recoverable Canadian traveller problem. In *Proc. of the International Conference on Management Science and Engineering*, (2004), pp.633–639.

[10] B. Su, Y.F. Xu, P. Xiao, and L. Tian. A risk-reward competitive analysis for the recoverable Canadian traveller problem. In *Proc. of the 2nd Conference on Combinatorial Optimization and Applications (COCOA)*, (2008), LNCS 5165, pp. 417–426.

[11] S. Westphal. A note on the k-Canadian traveller problem. *Information Processing Letters*, (2008), 106, pp. 87–89.

[12] Y.F. Xu, M.L. Hu, B. Su, B.H. Zhu, and Z.J. Zhu. The Canadian traveller problem and its competitive analysis. *Journal of combinatorial optimization*, (2009), 18, pp.195–205.

[13] H. Zhang and Y.F. Xu. The k-Canadian traveller problem with communication. In *Proc. of the 5th International Frontiers of Algorithmics Workshop (FAW-AAIM)*, (2011), LNCS 6681, pp. 17–28.